

Student Name: Ubaid Omarjee

Student Number: ST10315716

Module Code: PDAN8412/w

Submission Date: 23 September 2026

PART 1 – REPORT

Contents

1. Introduction
2. Dataset Selection and Justification
3. Analysis Plan
4. Data Acquisition and Spark Analysis
5. Data Quality Assessment and Cleaning
6. Exploratory Data Analysis
7. Feature Selection and Preparation
8. Model Architecture
9. Model Training
10. Model Evaluation
11. Confusion Matrix Interpretation
12. Hyperparameter Experimentation and Model Improvement
13. Example Prediction
14. Limitations
15. Conclusion
16. References

1. Introduction

This portfolio investigates whether recurrent neural networks can classify written text according to its author. The task is formulated as a multiclass text-classification problem in which a blog post is the input and the associated author identifier is the target class. The selected approach uses a Bidirectional Long Short-Term Memory (LSTM) network because text is sequential and the order and context of tokens can provide information useful for distinguishing writing patterns. TensorFlow's RNN text-classification guidance demonstrates the use of text vectorisation, embeddings and recurrent layers for sequence classification (TensorFlow, n.d.-a).

The analysis uses the Blog Authorship Corpus and follows an end-to-end analytical workflow: dataset acquisition, Spark-based inspection and cleaning, exploratory data analysis, feature preparation, model development, model training, evaluation and hyperparameter experimentation. The objective is not to claim that the model can identify any author in general, but to measure how effectively it can distinguish the ten selected authors represented in the final modelling dataset.

2. Dataset Selection and Justification

The selected dataset is the Blog Authorship Corpus. The Kaggle data card describes a corpus collected from blogger.com in August 2004, containing 681,288 posts from 19,320 bloggers and more than 140 million words (Kaggle, n.d.). The original corpus is therefore substantially larger than the assessment requirement of at least 10,000 records. The dataset is directly aligned with the problem because each post is associated with a blogger identifier.

The original research associated with the corpus demonstrates that blog writing contains differences in style and content that can be analysed in relation to author characteristics (Schler et al., 2006). This provides a reasonable basis for investigating author-related classification from text, while the present project uses a neural-network approach rather than reproducing the original study.

The dataset is appropriate for an LSTM because the principal predictor is natural-language text. Each observation contains a sequence of words and the author identifier acts as the class label. However, the dataset is historical and the original collection was produced in 2004, so its language and topics may differ from modern online writing (Kaggle, n.d.). In addition, author activity is highly uneven, creating a risk that a classifier could receive much more information about some authors than others. These issues motivated author selection, balancing, cleaning and stratified train/validation/test splitting.

3. Analysis Plan

The analysis was structured before model training. First, the dataset was downloaded and loaded into Apache Spark. The schema, record count and author distribution were inspected. Data-quality checks then identified null or empty text, duplicate records and differences in the number of posts contributed by authors.

A manageable working dataset was created by selecting ten authors with sufficient observations and retaining up to 1,000 posts per author. The text was then cleaned by removing URLs, converting text to lowercase, normalising whitespace, excluding posts shorter than 50 characters and removing duplicate author-text combinations. Exploratory analysis examined text length and final class distribution.

The final Spark dataset was converted to Pandas for TensorFlow/Keras modelling. Author identifiers were label encoded, and the data was divided into training, validation and test sets using stratification. TextVectorization was adapted on training text only and used to create fixed-length sequences. A baseline Bidirectional LSTM was then trained and evaluated using accuracy, precision, recall, weighted F1-score and a confusion matrix. A second configuration changed LSTM capacity, dropout and learning rate to test whether those changes improved generalisation.

4. Data Acquisition and Spark Analysis

Apache Spark was used for the required data analysis. The downloaded CSV contained the columns id, gender, age, topic, sign, date and text. Spark was used to inspect the schema and sample records before any modelling was performed. The Spark DataFrame contained 681,284 original records and 19,320 unique author identifiers.

Spark aggregations were used to calculate posts per author. The twenty authors with the largest numbers of posts showed clear differences in available observations, with the largest author having 4,221 posts compared with 1,351 for the twentieth author in the displayed top-twenty table. Spark's groupBy operation is intended for grouping a DataFrame so that aggregations can be performed on the groups (Apache Spark, n.d.).

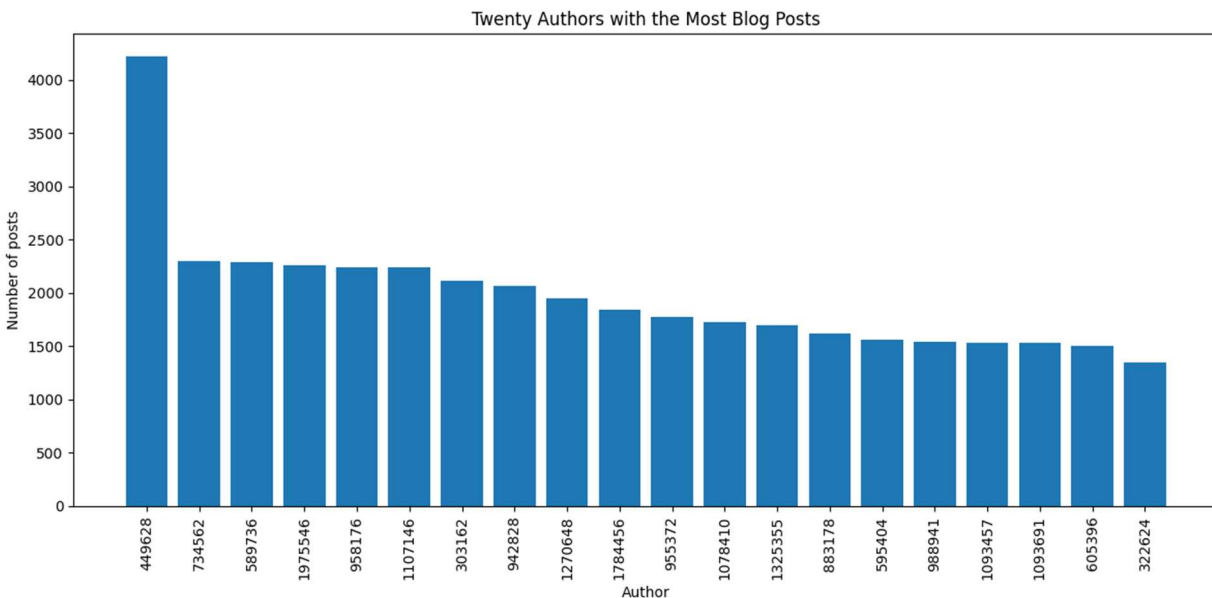


Figure 1: Twenty authors with the most blog posts in the original dataset.

The distribution confirms that the original dataset is not naturally balanced across authors. To make the modelling problem manageable and reduce the effect of severe class-size differences, the ten authors with sufficient observations were selected. The selection produced 23,504 records before limiting each selected author to 1,000 posts. The resulting working dataset contained exactly 10,000 posts, with 1,000 posts per author before the later quality filters.

5. Data Quality Assessment and Cleaning

The initial quality assessment found no null values in the author or text columns. However, 3,085 records were identified as having null or empty text, and 4,686 exact duplicate rows were identified. Records without usable text cannot provide meaningful evidence to a text classifier, while duplicate information can distort evaluation and cause the same textual pattern to be represented more than once.

The text cleaning procedure was deliberately moderate. URLs were removed because they may represent external links rather than an author's linguistic style. Text was converted to lowercase and repeated whitespace was normalised. Punctuation and other stylistic information were not aggressively removed because those characteristics may contain useful author-related information.

The 10,000-record balanced dataset was reduced to 8,548 records after the minimum text-length filter of 50 characters. A further 24 duplicate author-text combinations were removed, producing a final dataset of 8,524 records from ten authors. This approach is consistent with the broader principle that reliable model evaluation requires meaningful, non-duplicated examples in the evaluation data (Google for Developers, n.d.-a).

6. Exploratory Data Analysis

The final text-length statistics showed a count of 8,524 posts, a mean length of 581.18 characters and a standard deviation of 1,048.22 characters. The minimum length was 50 characters and the maximum was 30,549 characters. The distribution was strongly right-skewed, with many relatively short posts and a smaller number of very long posts.

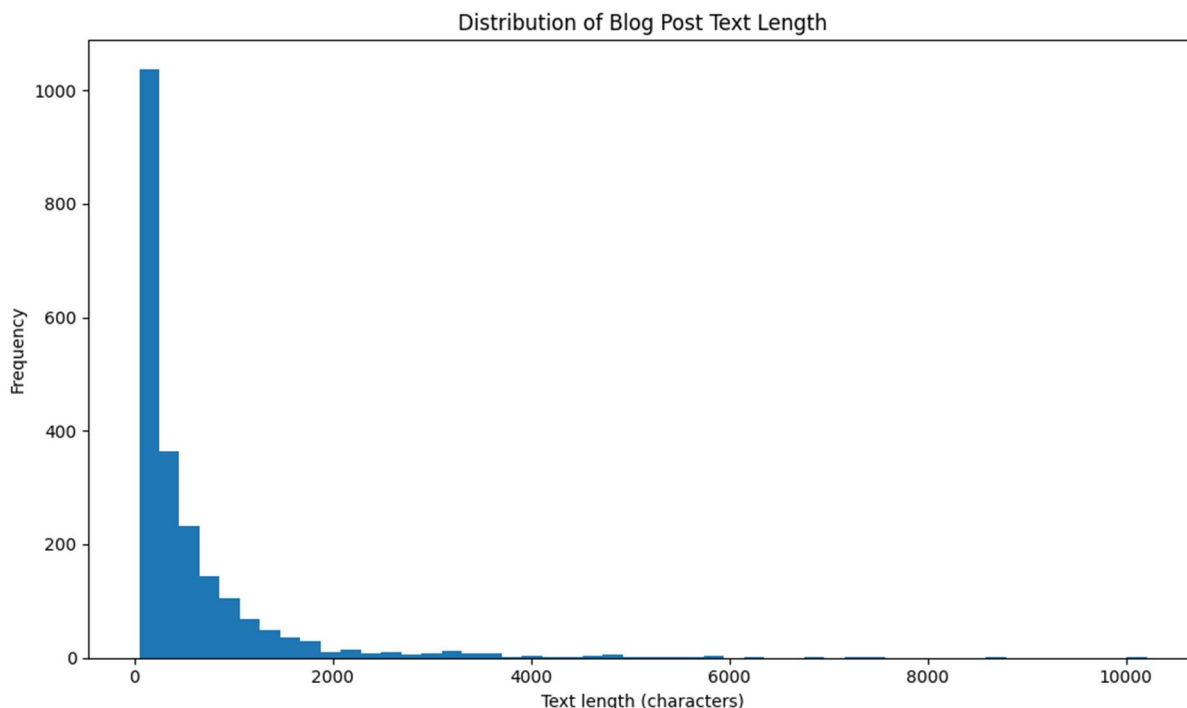


Figure 2: Distribution of cleaned blog-post length.

The wide variation in post length influenced the sequence representation used by the model. A fixed sequence length of 300 tokens was selected so that all model inputs had a consistent shape. Shorter posts

are padded and longer posts are truncated. This provides a practical computational compromise, although truncation means that some information from longer posts is not presented to the model.

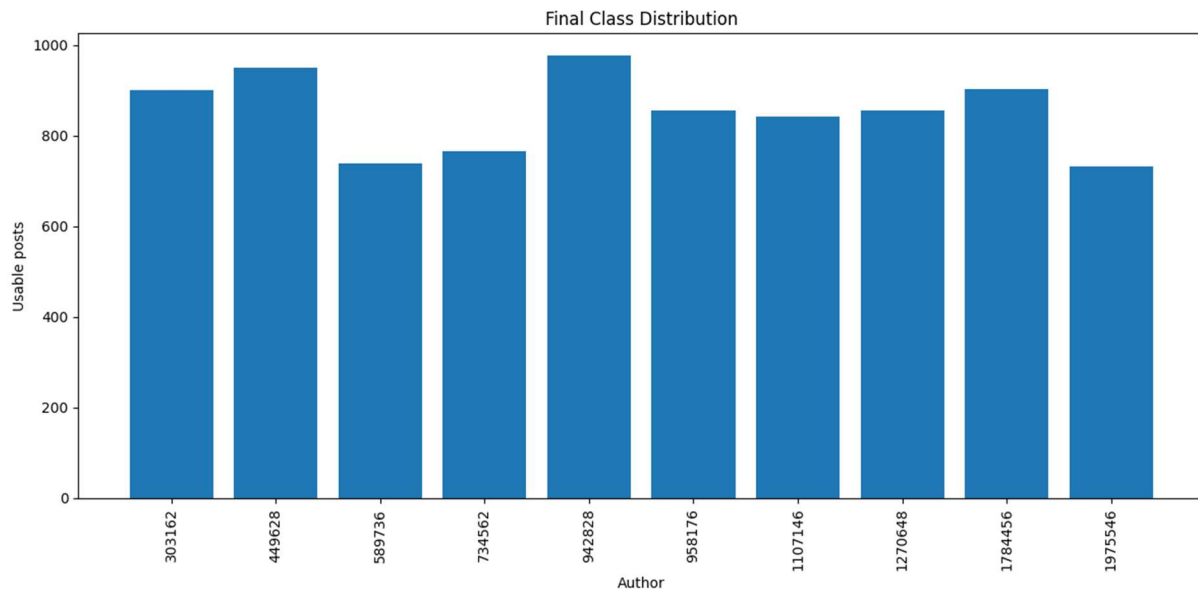


Figure 3: Final number of usable posts per selected author after cleaning.

The final class distribution was no longer perfectly equal because the short-text and duplicate filters affected authors differently. The final counts ranged from 733 posts for author 1975546 to 977 posts for author 942828. Despite this variation, all ten classes remained represented. Stratified splitting was therefore used to preserve the relative class distribution across the training, validation and test subsets.

7. Feature Selection and Preparation

The primary predictor was the cleaned blog-post text, while the author identifier was the target variable. The author identifiers were converted to numerical class labels using Scikit-learn's label encoding so that the neural network could operate on integer class targets.

The final 8,524 observations were split into 6,819 training records, 852 validation records and 853 test records. Stratification was used so that each partition retained a similar relative representation of the ten author classes. Separating training, validation and test data is important because the model should ultimately be assessed on examples that were not used for fitting or repeated model adjustment (Google for Developers, n.d.-a).

TensorFlow's TextVectorization layer was then used with a maximum vocabulary of 20,000 tokens and a sequence length of 300. The vectoriser was adapted using training text only. This prevents the validation and test sets from influencing the learned vocabulary and reduces the risk of information leakage. TensorFlow's text-processing documentation describes vectorisation as a standard step for converting raw text into a form suitable for neural-network modelling (TensorFlow, n.d.-c).

8. Model Architecture

The baseline architecture was TextVectorization → Embedding → Bidirectional LSTM → Dense → Dropout → Softmax. The embedding layer transforms token IDs into trainable dense representations. The Bidirectional LSTM processes the sequence in both directions, allowing information from earlier and later parts of the input sequence to contribute to the representation used for classification. TensorFlow's RNN documentation describes the bidirectional approach as a wrapper that propagates the input forward and backward through the recurrent layer (TensorFlow, n.d.-a).

The Keras LSTM layer provides the recurrent component used in the model. LSTM layers are designed to process sequence inputs while maintaining recurrent state information, making them suitable for sequential text representations (Keras, n.d.). A dense ReLU layer provides additional nonlinear processing, while dropout acts as a regularisation mechanism. The final softmax layer contains ten outputs, one for each selected author class.

Layer / Setting	Configuration	Purpose
TextVectorization	20,000-token vocabulary; 300 tokens	Convert text to fixed-length integer sequences
Embedding	128 dimensions	Learn dense token representations
Bidirectional LSTM	64 units	Learn forward and backward sequence patterns
Dense	64 units, ReLU	Further nonlinear representation learning
Dropout	0.50	Regularisation
Output	10 units, softmax	Produce probabilities for ten authors
Optimiser	Adam, learning rate 0.001	Optimise model weights
Loss	Sparse categorical cross-entropy	Multiclass classification loss

9. Model Training

The baseline model was trained for a maximum of 15 epochs with a batch size of 64. Early stopping monitored validation loss with a patience of three epochs and restored the best model weights. Early stopping is a recognised regularisation approach because it can stop training when validation loss begins to deteriorate rather than allowing the model to continue fitting the training data (Google for Developers, n.d.-b).

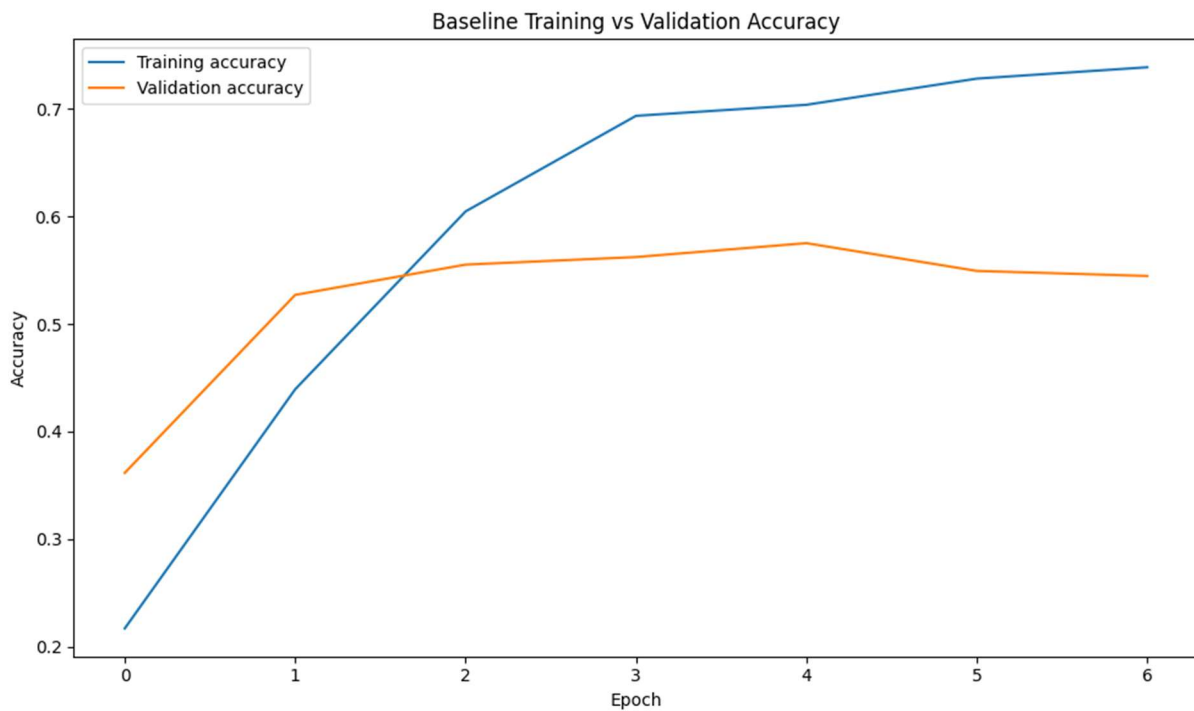


Figure 4: Baseline training and validation accuracy.

Training accuracy increased from 21.69% in epoch 1 to 73.87% by epoch 7. Validation accuracy increased from 36.15% to a peak of 57.51% in epoch 5, before falling to 54.93% and 54.46% in epochs 6 and 7. This divergence indicates that the model continued to fit the training data while validation performance stopped improving.

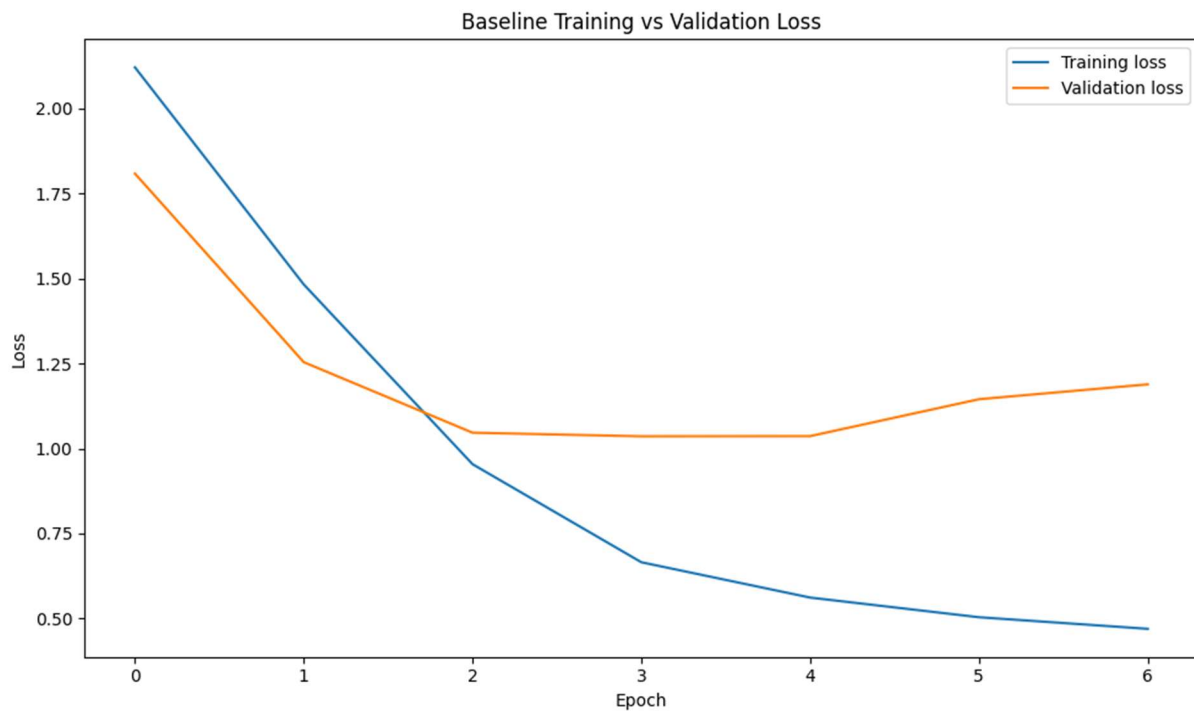


Figure 5: Baseline training and validation loss.

The loss curves show the same pattern. Training loss decreased continuously, while validation loss fell strongly at first and then began increasing after the early epochs. This is consistent with overfitting, where training performance continues to improve but performance on unseen examples deteriorates (Google for Developers, n.d.-b). The use of early stopping and restoration of the best validation-loss weights was therefore appropriate.

10. Model Evaluation

The baseline model was evaluated on the held-out test set of 853 observations. The test loss was 1.0796 and the model achieved 54.87% accuracy, 53.93% weighted precision, 54.87% weighted recall and 52.42% weighted F1-score. These metrics provide a more complete view than accuracy alone because they show how prediction errors are distributed across classes.

Metric	Baseline	Experimental
Test accuracy	54.87%	32.12%
Weighted precision	53.93%	18.78%
Weighted recall	54.87%	32.12%
Weighted F1-score	52.42%	22.98%

The per-author classification report shows substantial differences between classes. Author 942828 achieved the highest F1-score at 0.89, while authors 1784456 and 1270648 achieved 0.82 and 0.71 respectively. In contrast, author 589736 achieved an F1-score of only 0.03 and author 734562 achieved 0.14. This indicates that the model did not learn equally distinctive representations for all authors.

The variation is important when interpreting the overall 54.87% accuracy. A single overall score can hide poor performance for individual classes, so the classification report was necessary for a more detailed assessment.

11. Confusion Matrix Interpretation

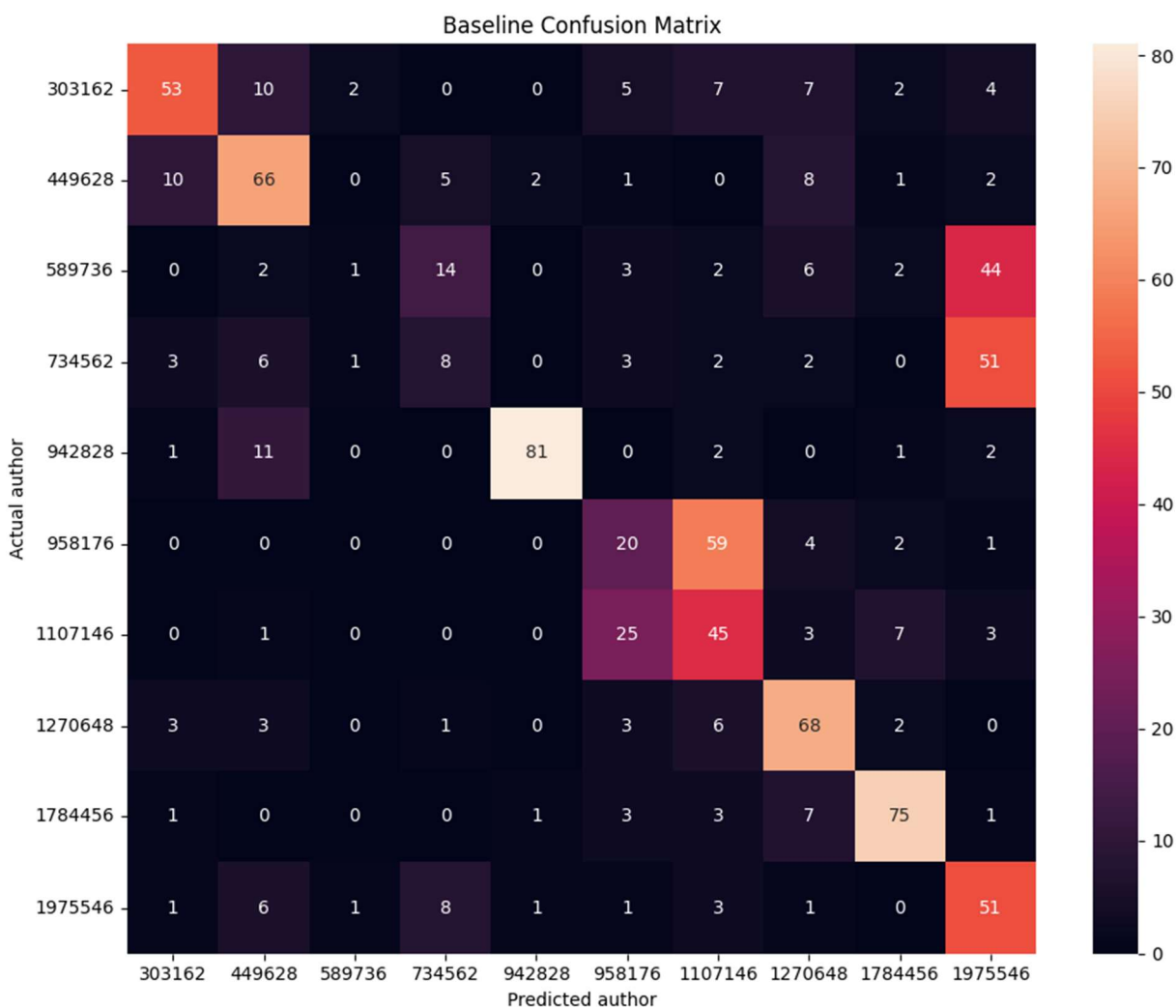


Figure 6: Baseline confusion matrix on the held-out test set.

The confusion matrix shows the distribution of correct and incorrect predictions across the ten author classes. Correct predictions appear on the diagonal. Author 942828 was classified particularly well, with 81 of 98 test observations correctly classified. Author 1784456 also had a strong diagonal result, with 75 of 91 observations correctly classified.

Several classes showed notable confusion. Of the 74 test observations belonging to author 589736, 44 were predicted as author 1975546. Similarly, 51 of the 76 observations belonging to author 734562 were predicted as author 1975546. Author 958176 also had substantial confusion with author 1107146, with 59 of 86 observations assigned to that class. These patterns help explain the low recall and F1-scores for some authors and suggest that their selected text representations were not sufficiently distinct for the baseline model.

12. Hyperparameter Experimentation and Model Improvement

A second model was trained to investigate whether changing the recurrent capacity, dropout and learning rate could improve generalisation. The baseline used 64 LSTM units, 0.50 dropout and a learning rate of 0.001. The experimental model used 128 LSTM units, 0.40 dropout and a learning rate of 0.0005. The embedding dimension remained 128.

The experimental configuration did not improve test performance. It achieved 32.12% accuracy, 18.78% weighted precision, 32.12% weighted recall and 22.98% weighted F1-score. These values are all below the corresponding baseline results. The validation-accuracy curves also show that the experimental model reached 49.53% by its third recorded epoch, while the baseline had already reached 55.52% in epoch 3 and later peaked at 57.51%.

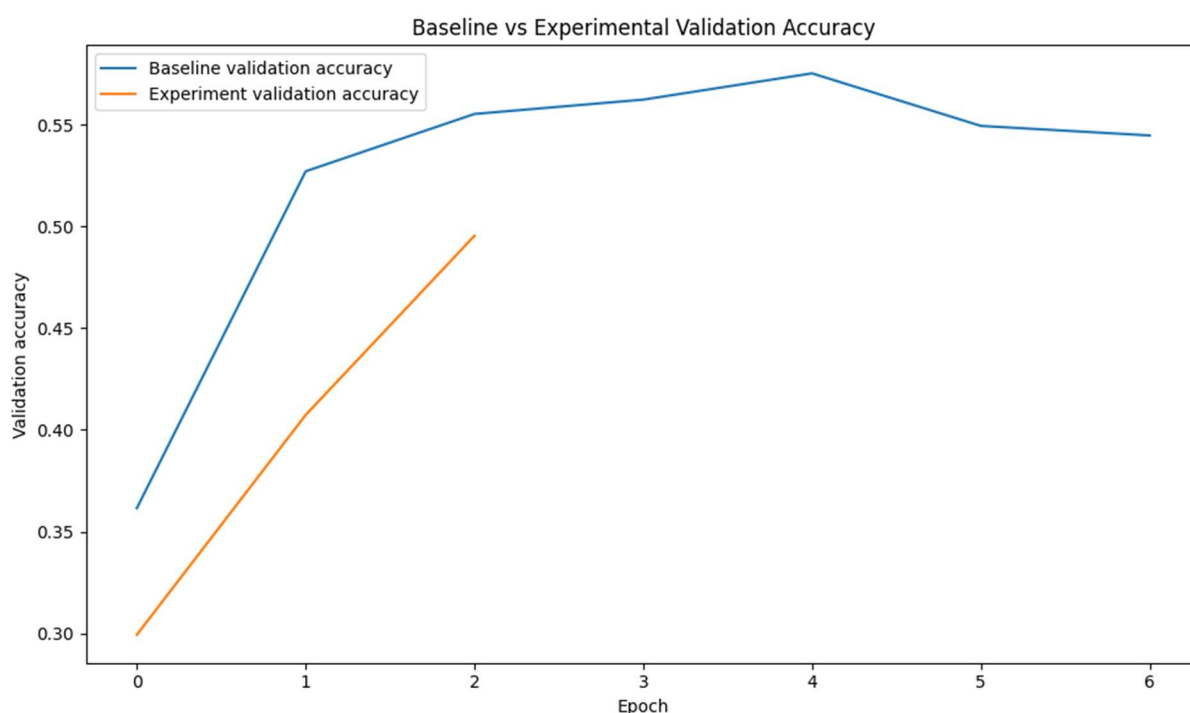


Figure 7: Baseline versus experimental validation accuracy.

The comparison demonstrates that increasing recurrent capacity did not automatically improve generalisation. The baseline model was therefore retained as the final model because it achieved the higher weighted F1-score on the unseen test data. The final selected model had 54.87% accuracy and a weighted F1-score of 52.42%.

13. Example Prediction

The final model was tested on an artificial text sample that was not part of the original dataset. The model predicted author 303162 with a probability of 0.8944, equivalent to approximately 89.44%.

This example demonstrates how the classifier can be applied to a new text input. The probability should not be interpreted as proof of authorship. It represents the model's estimated probability for the selected class

under the learned model and the distribution represented by the training data. New writing from authors outside the selected ten classes would not be appropriately represented by this multiclass model.

14. Limitations

Several limitations affect the interpretation of the results. First, the Blog Authorship Corpus is historical. The Kaggle description states that the posts were collected in 2004, so the vocabulary, topics and conventions may not represent current online writing (Kaggle, n.d.).

Second, the model was trained on only ten selected authors rather than the full population of 19,320 bloggers. The results therefore describe performance on the selected ten-class problem and cannot be generalised to arbitrary unseen authors.

Third, authorship classification can be influenced by topic and vocabulary. If particular authors repeatedly discuss particular subjects, a model may learn topical words as predictive signals in addition to genuine writing style. The present analysis did not perform a separate topic-control experiment, so this effect cannot be isolated from the reported results.

Fourth, the fixed sequence length of 300 tokens means that long posts are truncated. The final text-length statistics show a maximum length of 30,549 characters, so some posts contain considerably more information than the model receives.

Finally, neural-network results can vary with random initialisation and hardware/software conditions. A fixed random seed was used to improve reproducibility, but this does not guarantee identical results across all environments. The test results should therefore be treated as the measured performance of this implementation and configuration rather than a universal performance guarantee.

15. Conclusion

This analysis developed and evaluated a Bidirectional LSTM model for predicting the author of blog posts from their written content. Apache Spark was used for dataset loading, aggregation, author selection, cleaning, balancing and exploratory analysis, while TensorFlow/Keras was used for text vectorisation and neural-network modelling. The workflow resulted in a final cleaned dataset of 8,524 observations from ten author classes.

The baseline model achieved 54.87% test accuracy and a weighted F1-score of 52.42%. Training curves showed that training accuracy continued to increase after validation performance had peaked, providing evidence of overfitting and supporting the use of early stopping. The per-author results showed considerable variation, with some authors being much easier to identify than others.

The hyperparameter experiment, which increased LSTM capacity to 128 units, reduced dropout to 0.40 and lowered the learning rate to 0.0005, did not improve performance. Its test accuracy was 32.12% and weighted F1-score was 22.98%, so the baseline configuration was selected as the final model based on the measured test results.

Overall, the experiment provides evidence that an LSTM can learn author-related patterns within the selected historical blog dataset, but the results also demonstrate the limits of such a model. Performance varies substantially by author, the corpus is historical, the model uses only ten classes, and fixed-length text representation can discard information. The findings should therefore be interpreted as performance on this

specific dataset and modelling setup rather than as evidence that authorship can be reliably identified from arbitrary modern text.

16. References

- Apache Spark (n.d.) 'pyspark.sql.DataFrame.groupBy'. Available at:
<https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.groupBy.html> (Accessed: 23 September 2026).
- Google for Developers (n.d.-a) 'Datasets: Dividing the original dataset'. Machine Learning Crash Course. Available at: <https://developers.google.com/machine-learning/crash-course/overfitting/dividing-datasets> (Accessed: 23 September 2026).
- Google for Developers (n.d.-b) 'Overfitting'. Machine Learning Crash Course. Available at:
<https://developers.google.com/machine-learning/crash-course/overfitting/overfitting> (Accessed: 23 September 2026).
- Kaggle (n.d.) 'Blog Authorship Corpus'. Available at: <https://www.kaggle.com/datasets/rtatman/blog-authorship-corpus> (Accessed: 23 September 2026).
- Keras (n.d.) 'LSTM layer'. Available at: https://keras.io/api/layers/recurrent_layers/lstm/ (Accessed: 23 September 2026).
- Schler, J., Koppel, M., Argamon, S. and Pennebaker, J. (2006) 'Effects of Age and Gender on Blogging', Proceedings of the 2006 AAAI Spring Symposium on Computational Approaches for Analyzing Weblogs. Available at: <https://aaai.org/papers/0039-ss06-03-039-effects-of-age-and-gender-on-blogging/> (Accessed: 23 September 2026).
- TensorFlow (n.d.-a) 'Text classification with an RNN'. Available at:
https://www.tensorflow.org/text/tutorials/text_classification_rnn (Accessed: 23 September 2026).
- TensorFlow (n.d.-c) 'Text and natural language processing with TensorFlow'. Available at:
<https://www.tensorflow.org/tutorials/text> (Accessed: 23 September 2026).